

SERVICE ACTUEL · BUSINESS-FRONT-END

Interface métier de conversations, agents et applications

6 capacités analysées

services/business-front-end · TypeScript, React, Vite, Chakra UI 3, TanStack Query et contextes React, bun, Keycloak JS ; pas de Next ni Redux.

SPA destinée aux utilisateurs métier pour dialoguer avec des agents, configurer leur contexte et ouvrir les applications disponibles.

Lecture de code Pas de qualification runtime nouvelle

Domaines cible : Frontends, IAM, AI, Embeddings, MCP, Applications, Files, Query.

Périmètre de lecture

- Rapport 14 et instructions du composant.
- AuthProvider, users.access, clients conversations/agents/applications/fileSessions/corpora/toolboxes/datasets.
- Routes chat, agents et applications ; configuration runtime documentée.

Objets et cycle de vie

- Utilisateur et studio IAM, décision d'accès, agents/versions, conversations/messages.
- Documents/versions, sessions de fichiers, corpus, toolboxes/variables et applications.

Dépendances du service

- IAM/IdP, AI, Embeddings/corpus, MCP/tools, Applications, Files et Query.
- TanStack Query, React context et SSE/XHR.

Fonctionnalités et compatibilité à préserver

BFE-01 Cible non qualifiée

Admission à l'interface métier et contexte utilisateur

COMPORTEMENT OBSERVÉ

AuthProvider attend l'initialisation Keycloak et l'authentification puis charge utilisateur, permissions et espace studio. Seul system.canAccessFullInterface booléen vrai et non dégradé autorise l'application. Valeur absente/mal formée, erreur ou header de dégradation affiche une page de reprise ; le refus explicite possède une variante restreinte. L'ID studio est directement l'ID projet IAM. La langue utilisateur alimente i18n. La décision est rafraîchie périodiquement sans maintien permissif d'une dernière bonne réponse.

UTILISATEURS ET RÔLES

Utilisateur métier.

INTERFACES DANS LE CODE

AuthProvider ; getBusinessAppAccess ; readBusinessAppAccess ; accessView.

DONNÉES ET ÉTATS

AuthContext, utilisateur, studiold et décision allowed/restricted/unavailable.

DÉPENDANCES ET ÉCHANGES

IAM, IdP Keycloak et clients HTTP.

COMPOTEMENT CIBLE PROPOSÉ

Frontends TypeScript consomme IAM Rust via un adaptateur OIDC indépendant du fournisseur. Le contrat à trois états reste identique.

MIGRATION DE CETTE CAPACITÉ

Préserver la sémantique des IDs et la dégradation ; migrer les endpoints sans extraire un ID depuis le nom du projet.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un champ non booléen ou un IAM indisponible n'ouvre pas l'application. Après révocation, le prochain rafraîchissement ferme l'accès avec le bon message.

Références de l'analyse

[monorepo · services/business-front-end/app/src/providers/AuthProvider.tsx:17](#) Chargement identité, studio et admission.

[monorepo · services/business-front-end/app/src/entities/users/users.access.ts:10](#) Validation stricte et dégradation.

[monorepo · services/business-front-end/app/src/entities/users/users.access.ts:41](#) Trois états explicites.

BFE-02 **Cible non qualifiée**

Conversation avec un agent et historique**COMPOTEMENT OBSERVÉ**

Le client liste l'historique global ou par agent, charge une conversation et crée/continue une conversation par SSE/XHR en POST/PUT. La création peut cibler une version de contexte. Le détail utilise staleTime infini et une rétention de cache prolongée pendant le streaming, afin qu'une navigation hors du chat conserve l'historique déjà chargé. Renommage et suppression sont des commandes HTTP dédiées.

UTILISATEURS ET RÔLES

Utilisateur dialoguant avec un agent.

INTERFACES DANS LE CODE

getConversationsHistory ; getConversation ; createConversation ; continueConversation ; updateConversation ; deleteConversation.

DONNÉES ET ÉTATS

Conversation, messages, titre, agent/contextVersion et buffer client durant le flux.

DÉPENDANCES ET ÉCHANGES

AI, sessions conversationnelles, IAM et SSE.

COMPOTEMENT CIBLE PROPOSÉ

Frontends conserve le chat sur AI Rust, avec IDs de message/run, provenance des entrées et reprise de flux. Les effets outillés suivent leurs contrats d'approbation serveur.

MIGRATION DE CETTE CAPACITÉ

Conserver l'historique et la version de contexte choisie ; ajouter curseur durable et déduplication des fragments sans supposer qu'un flux client vaut un résultat final.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Quitter puis revenir pendant une réponse ne duplique ni prompt ni message. Reconnecter récupère l'état autorisé ; une génération ne peut agir avec des droits issus du texte du prompt.

Références de l'analyse

[monorepo · services/business-front-end/app/src/entities/conversations/conversations.service.ts:30](#) Cache du détail pendant streaming.

[monorepo · services/business-front-end/app/src/entities/conversations/conversations.service.ts:43](#) Création avec version et continuation.

[monorepo · services/business-front-end/app/src/entities/conversations/conversations.service.ts:55](#) Renommage et suppression.

BFE-03 Cible non qualifiée

Configuration des agents, corpus et outils

COMPORTEMENT OBSERVÉ

Le client liste les agents, lit un agent avec currentVersion optionnelle et crée/modifie/supprime des agents dans un projet. Les corpus de projet peuvent être filtrés par IDs. Les toolboxes sont listées et détaillées, et leurs variables sont lues puis enregistrées par PATCH. La configuration d'un agent rassemble ainsi instructions et ressources venant de plusieurs services ; les validations d'accès doivent rester serveur.

UTILISATEURS ET RÔLES

Auteur d'agent dans son espace ou projet.

INTERFACES DANS LE CODE

getAgents ; getAgent ; createProjectAgent ; updateAgent ; deleteAgent ; getProjectCorpora ; getToolboxVars ; saveToolboxVars.

DONNÉES ET ÉTATS

Agent/version, AgentForm, corpus référencés et variables d'outils.

DÉPENDANCES ET ÉCHANGES

AI, corpus/Embeddings, MCP/toolboxes, IAM et projets.

COMPORTEMENT CIBLE PROPOSÉ

AI/MCP/Embeddings Rust publient une révision de contexte qualifiée, référencent les éditions de corpus et bornent les outils. Une variable secrète n'est pas un paramètre réaffichable librement.

MIGRATION DE CETTE CAPACITÉ

Importer références et versions puis qualifier les permissions de chaque corpus/outil. Conserver brouillon et version publiée séparés.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Modifier un brouillon n'altère pas une conversation épinglée à une version. Un corpus ou outil refusé ne peut être ajouté par manipulation du payload.

Références de l'analyse

[monorepo · services/business-front-end/app/src/entities/agents/agents.service.ts:20](#) Version de contexte et CRUD par projet.

[monorepo · services/business-front-end/app/src/entities/corpora/corpora.service.ts:8](#) Sélection des corpus.

[monorepo](#) · [services/business-front-end/app/src/entities/toolboxes/toolboxes.service.ts:38](#) Variables des outils.

BFE-04 Cible non qualifiée

Pièces jointes et sessions de fichiers

COMPORTEMENT OBSERVÉ

Le service client crée une session puis téléverse un fichier en multipart avec AbortSignal optionnel. Le même contrat est utilisé par les pièces jointes du chat et les formulaires d'agent. Une route DELETE retire un fichier de la session par nom. Le navigateur ne démontre pas à lui seul le scan, les limites ni la rétention, qui relèvent des serveurs.

UTILISATEURS ET RÔLES

Utilisateur attachant un document et auteur d'agent.

INTERFACES DANS LE CODE

`createFileSession` ; `uploadSessionFile` ; `deleteSessionFile`.

DONNÉES ET ÉTATS

Session de fichier, upload local, nom et annulation de requête.

DÉPENDANCES ET ÉCHANGES

AI sessions, Files et stockage serveur.

COMPORTEMENT CIBLE PROPOSÉ

Files/AI Rust fournissent upload borné, quarantaine et références immuables ; les corpus publiés suivent WAP. Le navigateur distingue transfert, validation et disponibilité pour le modèle.

MIGRATION DE CETTE CAPACITÉ

Conserver le parcours multipart et les erreurs, puis ajouter IDs stables de fichier/édition et état de validation. Ne pas assimiler annulation HTTP à suppression de l'objet déjà reçu.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Annuler au milieu puis réessayer ne laisse pas de document utilisé sans validation. Un fichier d'une autre session reste inaccessible même si son nom est connu.

Références de l'analyse

[monorepo](#) · [services/business-front-end/app/src/entities/fileSessions/fileSessions.service.ts:4](#) Session et upload.

[monorepo](#) · [services/business-front-end/app/src/entities/fileSessions/fileSessions.service.ts:14](#) Suppression partagée entre chat et formulaire.

BFE-05 Cible non qualifiée

Documents générés, graphiques et export de données

COMPORTEMENT OBSERVÉ

Le client consulte un document généré avec version optionnelle, demande les formats disponibles et télécharge le document. Il peut télécharger un graphique lié à un message. Un autre service envoie une requête d'export dataset avec options de séparateur/quoting et reçoit un blob. Les datasets du projet sont listés pour les usages métier. Le contrat actuel doit être enrichi pour prouver l'édition qui a servi au résultat.

UTILISATEURS ET RÔLES

Utilisateur métier qui consulte ou exporte une réponse.

INTERFACES DANS LE CODE

getConversationDocument ; getConversationDocumentFormats ; exportConversationDocument ;
downloadConversationChart ; downloadDataset.

DONNÉES ET ÉTATS

Document/version, graphique de message, blob téléchargé et requête d'export.

DÉPENDANCES ET ÉCHANGES

AI documents, Query/data-serve, Files et IAM.

COMPORTEMENT CIBLE PROPOSÉ

AI/Query/Files Rust lient chaque résultat à ses éditions d'entrée et à son reçu. EXPORT est un effet audité et autorisé ; il ne crée pas un neuvième kind de dataset.

MIGRATION DE CETTE CAPACITÉ

Préserver formats et noms d'usage, ajouter provenance et reprise du reçu, reconstruire les droits au moment d'export.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Après révocation, ni un ancien lien ni un cache ne permet un nouvel export. Une réponse et son export pointent vers la même version documentaire et les mêmes éditions de données.

Références de l'analyse

[monorepo · services/business-front-end/app/src/entities/conversations/conversations.service.ts:77](#) Document et version.

[monorepo · services/business-front-end/app/src/entities/conversations/conversations.service.ts:90](#) Formats et téléchargement.

[monorepo · services/business-front-end/app/src/entities/datasets/datasets.service.ts:10](#) Export de requête et paramètres.

BFE-06 Cible non qualifiée

Catalogue d'applications et shell métier au runtime

COMPORTEMENT OBSERVÉ

Le client charge les applications agents, conserve uniquement les statuts DEPLOYED ou WARNING puis les trie par nom. Le détail d'application possède sa propre clé de cache. Le router compose chat, agents et applications. La configuration est injectée via window avant fallback Vite, et la couche HTTP déballe déjà response.data. Le thème, les composants Chakra 3 et les contextes sont propres à cette SPA.

UTILISATEURS ET RÔLES

Utilisateur ouvrant une application publiée et opérateur de l'interface.

INTERFACES DANS LE CODE

getApplications ; getApp ; AppRouter ; config/env.ts.

DONNÉES ET ÉTATS

Liste d'applications disponibles, détail, configuration publique et cache.

DÉPENDANCES ET ÉCHANGES

Applications/app-management, IAM et assets publics.

COMPORTEMENT CIBLE PROPOSÉ

Frontends reste TypeScript avec Applications Rust. Disponibilité opérationnelle, droit de découverte et droit d'ouverture sont des propriétés séparées ; une application déployée n'est pas automatiquement visible.

MIGRATION DE CETTE CAPACITÉ

Conserver filtrage/tri et compatibilité runtime config ; qualifier liens, états de chargement, langues et accessibilité. Retirer le couplage IdP JVM sans changer les usages.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une application supprimée ou révoquée disparaît après invalidation et son lien direct est refusé côté serveur. Le même artefact frontend fonctionne dans deux environnements avec configuration publique injectée.

Références de l'analyse

[monorepo](#) · [services/business-front-end/app/src/entities/applications/applications.service.ts:20](#) Filtrage de statut et tri.

[monorepo](#) · [services/business-front-end/app/src/entities/applications/applications.service.ts:36](#) Détail.

[monorepo](#) · [services/business-front-end/app/src/AppRouter.tsx:8](#) Composition des routes.

[monorepo](#) · [services/business-front-end/AGENTS.md:44](#) Configuration publique au runtime.

Architecture et exploitation du service cible

Plan de contrôle Rust proposé

- SPA TypeScript conservée ; contexte/version d'agent, droits et provenance visibles.
- Le cache navigateur ne valide ni publication ni effet d'un outil.

Plan de données proposé

- Chat streamé, pièces jointes, rendu documentaire et exports bornés côté serveur.

Découpage du core Rust

- Aucun core métier navigateur Rust ; consommateurs TypeScript des APIs Rust AI/IAM/Files/Applications.

Identité, autorisation et isolation

- Admission stricte booléenne non dégradée, sans maintien permissif.
- Contrôler à nouveau l'accès au document, corpus, outil et export sur serveur.
- Ne pas exposer variables secrètes ou résultats de sessions non autorisées dans les caches.

Exploitation et reprise

- Config window au runtime avant fallback Vite, build bun et Chakra 3.
- Les conversations en cache pendant streaming requièrent invalidation correcte au logout/changement de contexte.
- Aucun parcours UI ou test en navigateur exécuté dans cet audit.

Séquence de migration

- Garder le contrat d'admission et remplacer l'adaptateur Keycloak.
- Versionner agents et provenance des corpus.
- Ajouter reprise du stream, reçus d'export et état d'upload validé.
- Qualifier personas, langues, focus/clavier et navigation entre chat et applications.

Écarts et décisions encore ouverts

- Reprise exacte d'un flux de conversation après perte réseau à qualifier.
- La protection et rétention des fichiers relèvent des services ; elles ne se déduisent pas des fonctions upload.
- Les effets d'outils doivent être classés selon leur niveau d'approbation ; le prompt utilisateur ne suffit pas à autoriser toute action.

Références de l'analyse

[monorepo](#) · [services/business-front-end/app/src/providers/AuthProvider.tsx:17](#) Admission et contexte.

[monorepo](#) · [services/business-front-end/app/src/entities/conversations/conversations.service.ts:30](#) Conversation et durée de cache.